

Optimisation and Rice

Every optimiser is an approximation. Sound or complete, never both — and register allocation is NP-hard.

[← CC 08 · Self-Compilation](#)

WHAT AN OPTIMISER CLAIMS

"This program means the same, and costs less." Both halves are about **behaviour**.

An optimisation replaces a program by another, claiming they are equivalent on every input and the second is cheaper. Equivalence is a property of behaviour, so by Rice it is undecidable in general. Cost is a metric, and metrics are approximations too.

Hence every optimiser answers a decidable stand-in: not "is this code dead?" but "is there a path in the text to it?"; not "is this value constant?" but "is it assigned once, from a literal?". Sound one way, incomplete the other.

WHY SELFIE DOES NOT

Not because it is hard to add. Every optimisation is a place where the compiler's output stops being a transcription of the source, and the class is about the transcription.

Constant folding and dead code elimination.

```
// before
x = 6 * 7;
if (0) y = 1;
while (c != 0) c = c + 2;
z = 2;    // never read again
// after folding and dead code
x = 42;
while (c != 0) c = c + 2;
```

Folding. $6 * 7$ is a term with two literal factors: the compiler can evaluate it and emit one addi. Sound, because the machine's arithmetic and the compiler's are the same 64-bit wrap.

Dead code. `if (0)` never runs; the text says so. `z = 2` is dead if nothing reads `z` afterwards, and "afterwards" is a path in the text, not an execution. The loop stays: removing a non-terminating loop changes behaviour.

Each decision is about the text, standing in for a fact about behaviour. The contract is that the stand-in is sound; its quality is how often it fires.

Register allocation is graph colouring, and graph colouring is NP-hard.

Selfie's stack of seven temporaries is the simplest allocator there is, and it fails at nesting depth eight. A real allocator asks which values are *live* at the same time; two that never are can share a register.

A node per value, an edge between values live simultaneously. Colouring the graph with k colours, no edge joining two of one colour, is allocation with k registers. Chaitin, 1981. And k -colouring is NP-complete: one more value, k times the colourings.

Liveness itself is a property of behaviour. The allocator uses the text's paths as the proxy, so the graph is an approximation before the colouring starts.

WHICH IS WHERE WEEK 11 BEGINS

Decidable, and unaffordable in the worst case. Compilers colour greedily, spill to memory when they fail, and get within a few percent of optimal on real code. Hardness in the worst case, structure in the typical one.

Sound but incomplete, or complete but unsound. Choosing is the discipline.

SOUND, INCOMPLETE

Everything it says is true; it does not say everything. A type checker. A dead-code pass that only removes what the text proves unreachable. A model checker's "no" up to k. Compilers must be here: an unsound optimisation is a miscompilation.

COMPLETE, UNSOUND

It finds everything, and some of what it finds is not there. A linter that warns on every suspicious pattern. A fuzzer's crash that is a bug in the harness. Useful for humans, fatal for a compiler.

Rice forbids sound and complete together for any interesting property. Every tool in the workshop around selfie is a stated choice: rotor and bitme sound within a bound, buzzr complete about crashes it sees, the type checker sound about pointers. The next three weeks build the first of these.

ASSIGNMENT

array-allocation: arrays in the data segment and on the stack.

- 1 Specify. Extend the grammar so that a variable declaration may be `type identifier [ integer ]`, globally and locally.
- 2 Allocate. Globally: n words in the data segment, and the symbol table records the size. Locally: n words in the frame, so the prologue's `addi sp, sp, -locals` grows and offsets from `s0` shift.
- 3 Access. Last week's `a[i]` now has two cases: the identifier is a pointer, or an array, whose address is its location rather than its value. The symbol table must say which.
- 4 `./grader/self.py array-allocation, self-compile`, and use a global array in `selfie.c`.

WATCH

Bounds are not checked: `a[n]` on an array of n words reads the neighbour. That is C, and week 12 is the first week that can find such an access for you.

Listen, then read.

LISTEN

[Strauss · Ariadne auf Naxos](#) — live, 1966, Régine Crespin. The Prologue: the richest man in Vienna orders the opera seria and the comedy played at the same time, so that the fireworks start on schedule. An optimisation under a deadline, with the Composer insisting that the meaning be preserved. It is.

READ

[Engineering a Compiler](#) — Cooper and Torczon: the optimising middle of a compiler, data-flow analysis and all, every pass a syntactic stand-in for a semantic fact. And, more technical, [Classes of Recursively Enumerable Sets and Their Decision Problems](#) — Rice, 1953: the theorem of the week, in the paper that proved it.